

U

O

W

CSIT314/ISIT950/MICT950 Software Development Methodologies

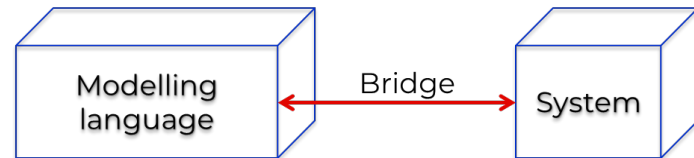
Advanced Unified Modelling Language



UNIVERSITY
OF WOLLONGONG
AUSTRALIA

Modelling Language

- High level designing [rather than low-level code details]
 - A standardised and well-defined modelling language
 - clear syntax and semantics
 - Enables abstraction and visualisation of complex systems
 - Simplifies systems by highlighting elements and representing them visually (e.g., diagrams)
 - Bridges high-level design and practical implementation
 - Connects conceptual models (e.g., class diagrams) with actual coding



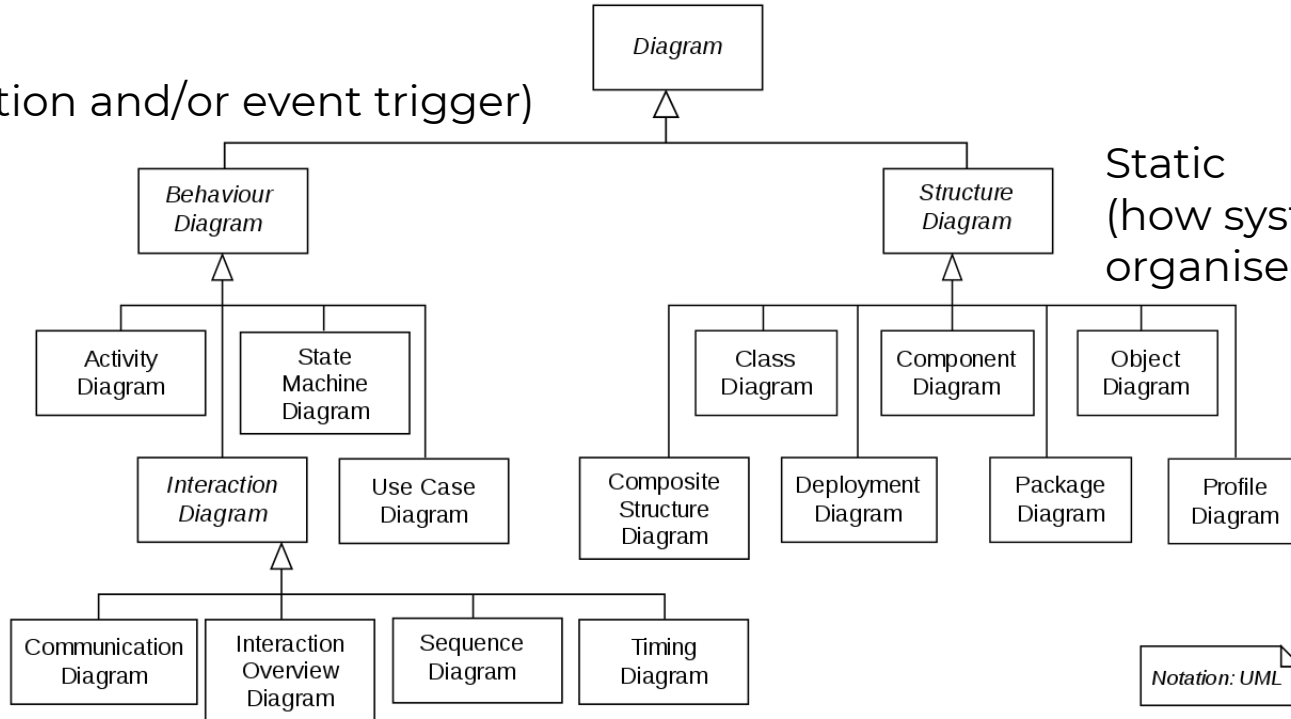
UML

- Unified modelling language (UML) is a general-purpose modelling language that is used to visualize the design of a system
 - how the system will run
 - individual components and their interaction
 - external user interface



UML [how to draw]

Dynamic
(user interaction and/or event trigger)



Static
(how systems being organised)

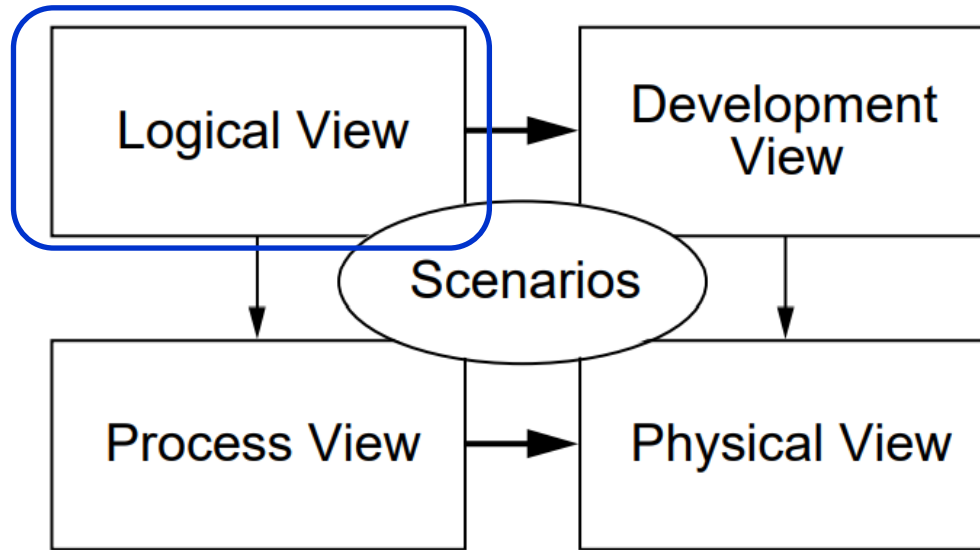
Notation: UML



“4+1” View [what to draw/show]

End-user
Functionality

Programmers
Software management



Integrators
Performance
Scalability

System engineers
Topology
Communications

- Various ‘stakeholders’
- Functional and non functional requirements
- Conceptual and Physical
- 1 is for user case view, or known as scenario (user requirement)
- Each view with different diagrams

Diagrams and views

- Logical views :
 - Class, object, package, state
- Process views :
 - Sequence, communication, activity
- Development views :
 - Component
- Physical views :
 - Deployment



Conceptual and physical

- Logical View and Process View
 - **conceptual** level
 - used from analysis to design
 - the logical view is concerned with the functionality that the system provides to end-users
 - the process view deals with the dynamic aspects of the system, how they communicate, and focuses on the runtime behavior of the system
- Development and Physical View
 - **physical** level (software and hardware)
 - represent the application components as built and deployed



Use case view

- Use Cases describe the non/functional requirements of a system
 - one or more Actors interacting with one or more Use Cases
 - Actor is a user or external system
 - Use Case is a typical interaction between an actor and the system
 - Use Cases specify the behavior from user's point of view,
 - As well as non-functional requirements (Performance, Security, ...)



Example

Use Case: User Authentication

Actor: Bank Customer

Description: The user wants to log in to the mobile banking application using their credentials.

Basic Flow:

The user opens the mobile banking application.

The user enters their username and password.

The application validates the user's credentials against the bank's authentication server.

If the credentials are valid, the application grants access to the user's account.

The user is redirected to the main dashboard of the mobile banking application.

Alternative Flows:

If the user enters invalid credentials:

Then blabla.

If the user forgets their password:

Then blabla

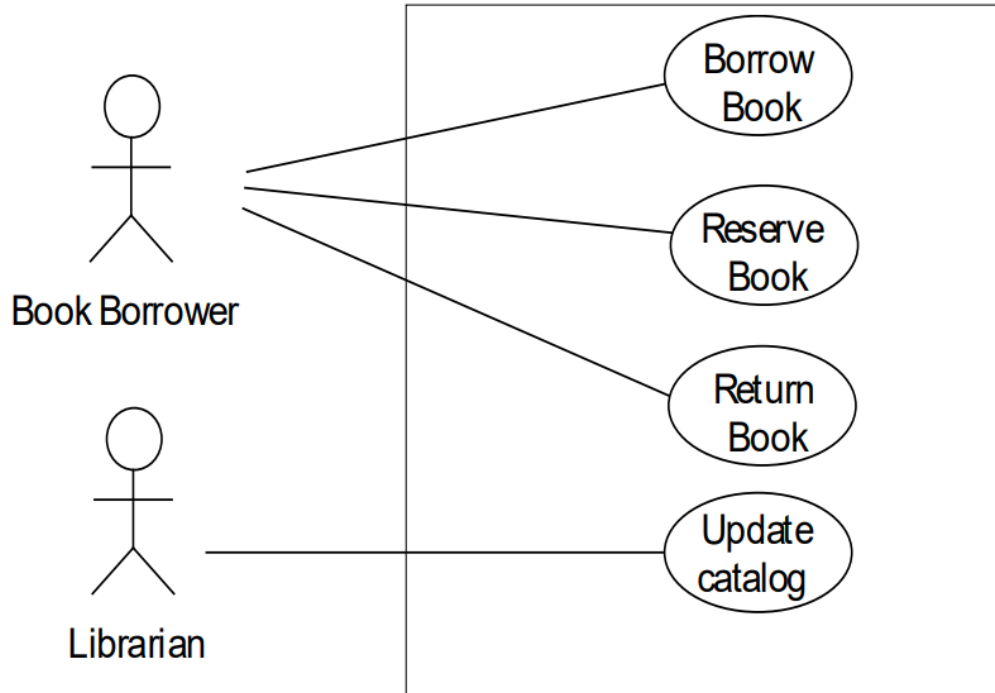


Use case vs user story

- **Level of Detail:** Use cases are typically more detailed and structured, providing a comprehensive view of system behavior. User stories, on the other hand, are concise and simple from users' side.
- **Format:** Use cases follow a standard format with specific sections, while user stories are often written in a more informal, conversational style.
- **Scope:** Use cases may cover multiple scenarios related to a particular feature or functionality, while user stories focus on individual user needs or goals.



Use Case Diagram

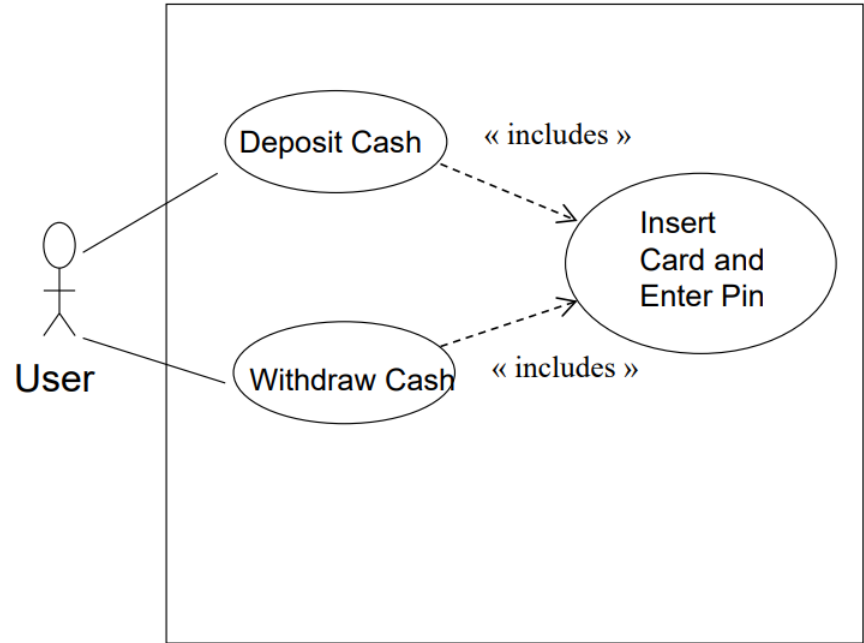


- **Visualises use cases (system functionalities)**
- Actors and Use Cases
 - 1:1
 - 1:m
 - m:m
- Each **oval** represents one use case

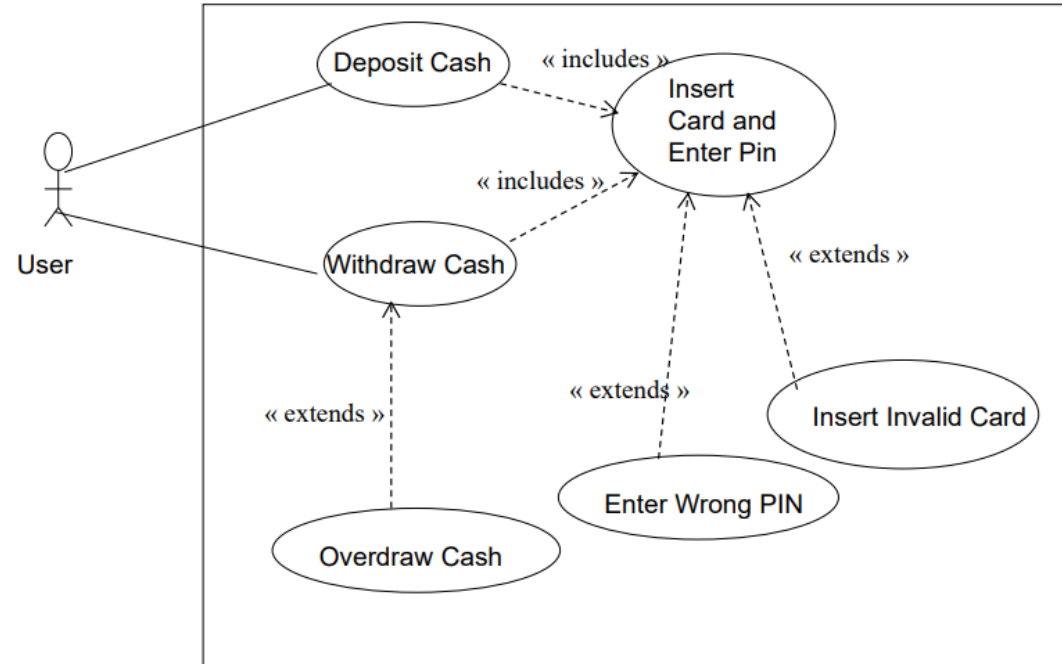


Use Case Diagram

- Use case depends on each other (such as sequence)
 - We need Relationship
- **Includes** Relationship
 - compulsory of Use Cases



Use Case Diagram

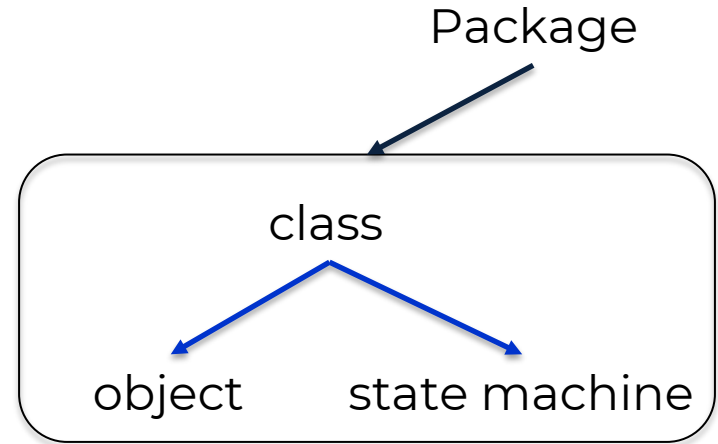


- **Extends** Relationship
 - optional of Use Cases
 - alternative courses of the same interaction
 - only occur under certain conditions



Logical view

- the organization of software components, their relationships, and the flow of data or control between them
- Main diagrams:
 - **class diagrams:** model the system
 - **package diagrams:** logically group different class diagrams
 - Optional diagrams :
 - Object: explain the class relationship via instances
 - **State Machine:** explain *internal states* of a specific class



UML class symbol

Person (class name)

empty

- Class symbols are the building blocks of class diagrams
- The most basic class symbol is a box with the class name
- Adding in more details may be needed at later stages
 - Attributes, or data types
 - Methods
 - The level of details depends on the related class diagram



UML class symbol

Person

- name:String
- age:int
- height:int
- rate:int {read only}
- + getName():String
- + setName():void
- + getHeight():int
- + getAge():int
- + setAge():void
- + main():void

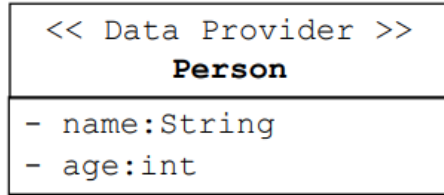
Prefix attributeName: Data type Prefix
methodName(): Return data type

Prefixes:

- + : public attributes and methods
- : private attributes and methods
- # : protected attributes and methods
- {read only}: constant
- __ (underline): static attributes and methods

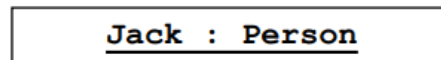
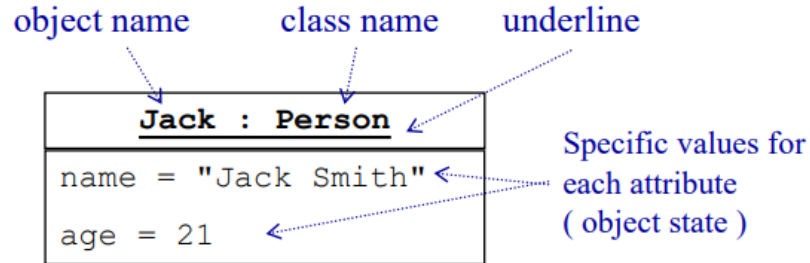


UML object symbol



A class

instantiation



A simplified UML representation
when state is not important



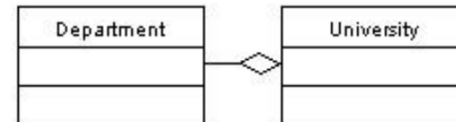
Logical View: class relationships

- Relationships between objects: [**more later**]
 - Aggregation: one object is a part of another object.
 - Composition: cannot exist independently
 - Inheritance: one object is a more specialized version of another object. This is also called specialization.
 - Association: The objects need to communicate (basically everything else - the default option)



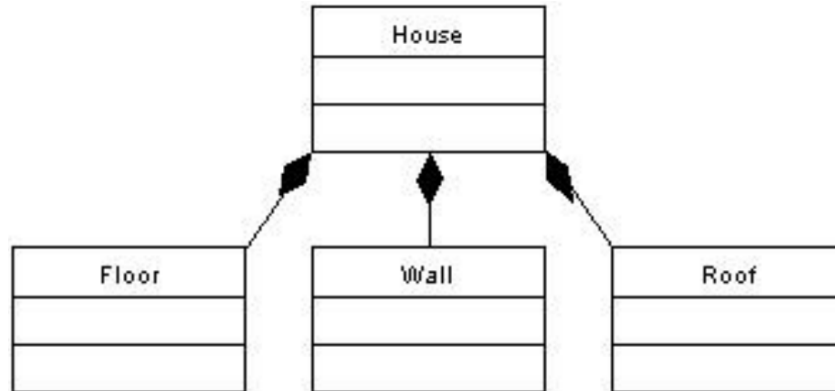
Aggregation

- an 'assembly-component' (i.e. an "is-a-part-of") relationship
- assembly object is a single object made up of component objects.
 - a department IS PART OF a university
- These things can also exist independently



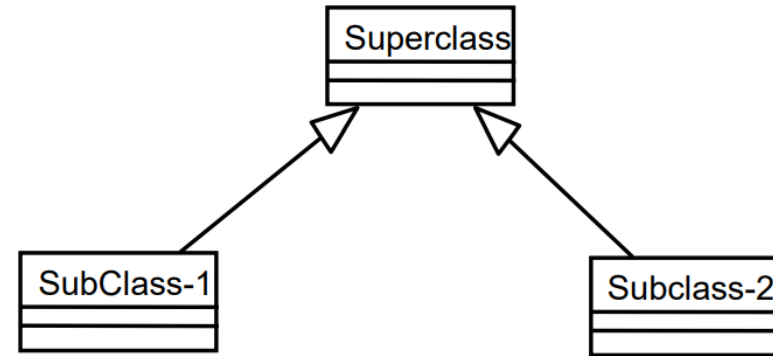
Composition

- ____ is composed of ____
 - E.g. a house is composed of a floor, roof and 4 walls
 - if the house is destroyed, then the other things are also destroyed



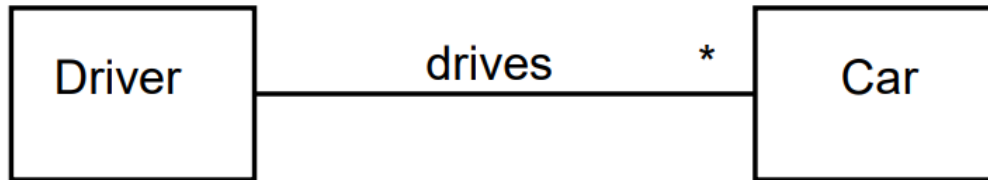
Inheritance

- Generalization/specialization
 - subclasses describe properties that are specializations of a superclass
 - an "is-a" relationship
 - all properties of superclass are inherited by subclass (attributes, operations and associations)



Association

- Associations are the most generic relationships.
 - Any relationships which do not fit into the other categories are association.
 - These relationships often are of the type 'has-a', 'uses', 'communicates with' or 'makes requests of'



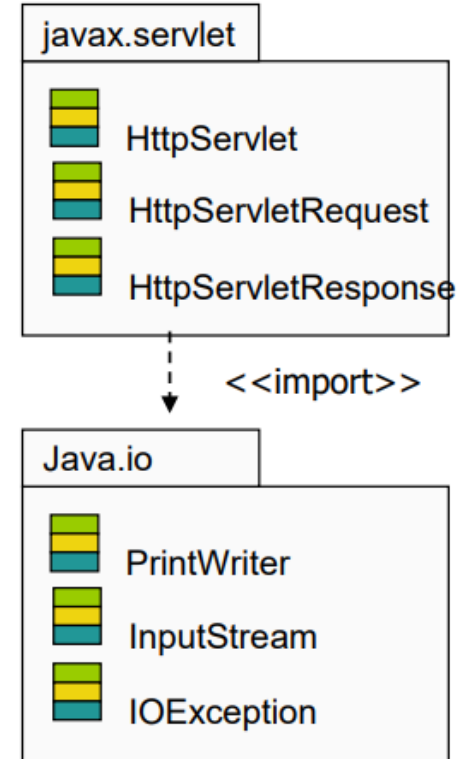
Logical View: Package diagram

- Similar to Java packages (or their equivalents in other language)
- It simply identifies a group of classes with a namespace
- Typically shows the public classes only

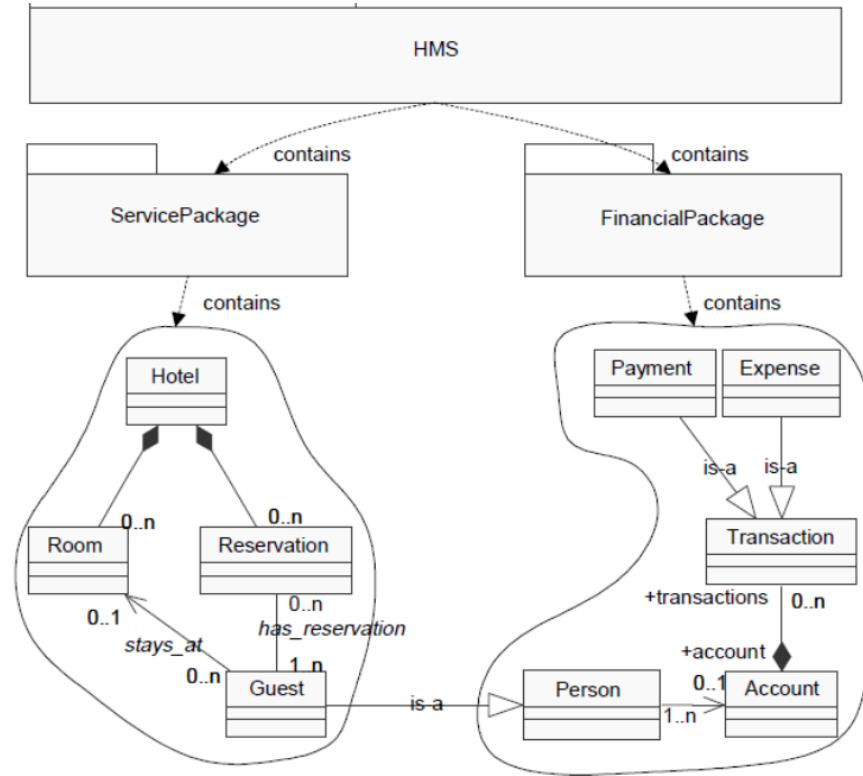


Package diagram

- Simplify UML diagrams by grouping related elements into a single higher-level element
 - such as **diagrams, classes, or even other packages**
- A dependency relationship
 - if a package is changed, the dependent package also could have to be modified



Package diagram benefit



Package diagram benefit

- **Modularization:** Package diagrams help in breaking down a system into manageable modules based on related functionality.
- **Hierarchical Representation:** They visually represent the nesting of packages, showing the system's architecture at different levels of abstraction.
- **Dependency Management:** Package diagrams illustrate dependencies between packages, aiding in effective management and reducing coupling.
- **Reusability:** Packages promotes reusability and enforcing modularity and separation of concerns.
- **Documentation and Communication:** They serve as visual documentation, facilitating communication and collaboration among stakeholders.



State diagram

- A dynamic model that shows the different states (status) that a single object during its lifetime in response to events (triggers).
- What is a state?
 - It is a set of values (of attributes) that describe an object at a specific point in an object's life
 - student's program, student's semester, etc. determine a state of the Student object
 - Some of the objects in the system are quite dynamic in that they pass through a variety of states over their existence
 - A student can change from “new” to “current” to “former

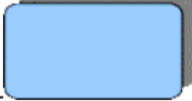


State diagram

- What is an Event (trigger)?
 - something takes place at a certain point and changes a value(s) that describe an object (the state of an object)
 - the student graduates from the university is an event (that changes the Student's state from "current" to "former")
 - concurrent events: two events that are related only by coincident and have no effect on each other
 - 2 planes take off at the same time



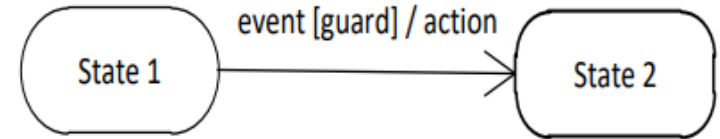
State diagram

Term and Definition	Symbol
A State Is shown as a rectangle with rounded corners.	
An Initial State Is shown with a small filled in circle	
A Final State Is shown with as a circle surrounding a small solid filled-in circle. This represents the completion of activity.	
An Event Is an occurrence that triggers a change in state. It is used to label a transition.	Event Name
A Transition Indicates that an object in the first state will enter the second state. Is triggered by the occurrence of the event labeling the transition. Is shown as a solid line.	



Transitions

- A relationship between 2 states
 - an object (from one state) will perform certain actions and enter the other state when a specific event occurs
- Source State
 - object receives the event and then it may “fire”
- Event Trigger
 - the event whose reception by the object makes it eligible to “fire”
- Guard Condition - An optional expression to see if it should “fire”.
 - "when you go out in the morning (event), if the temperature is below freezing (Guard condition), then put on your gloves (actions)" and move to next state.
- Action
 - an executable atomic computation
- Target State
 - the state that is active after the completion of the transition



Example

- Different states of a student

